

Sistema de Monitoramento e Controle *Closed Loop* para Aplicações uRLLC em Redes de Transporte 5G

Aluizio Nicodemos¹, Ygor Correa¹

¹Programa de Pós-Graduação em Tecnologia da Informação (PPGTI)
Instituto Federal da Paraíba (IFPB) – Campus João Pessoa
João Pessoa – PB – Brasil

nicodemos.aluizio@academico.ifpb.edu.br, correa.ygor@academico.ifpb.edu.br

Abstract. *Fifth-generation (5G) transport networks must simultaneously support latency-sensitive traffic (uRLLC) and high-throughput traffic (eMBB). This work presents the design, implementation and evaluation of a Closed Loop monitoring and control system that keeps the end-to-end latency of uRLLC flows within a 5ms budget in the presence of concurrent eMBB congestion. The system was emulated in Mininet over a four-router “diamond” topology, using Scapy for one-way-delay (OWD) measurement of TCP uRLLC traffic, iperf3 for eMBB traffic generation, and Linux tc/HTB for dynamic queue prioritization. Experimental results show that, without any control mechanism, a bandwidth-limited transport link degrades uRLLC latency from a baseline of $\approx 8\text{ms}$ to over 1.8s under eMBB congestion (peak of 4.7s); with the proposed dynamic Closed Loop, the same congestion scenario is automatically detected and mitigated within a few control cycles, restoring latency to baseline levels.*

Resumo. *Redes de transporte 5G precisam atender simultaneamente tráfego sensível à latência (uRLLC) e tráfego de alto volume de dados (eMBB). Este trabalho apresenta o projeto, a implementação e a avaliação de um sistema de monitoramento e controle Closed Loop capaz de manter a latência fim-a-fim de fluxos uRLLC dentro do limiar de 5ms mesmo sob congestionamento causado por tráfego eMBB concorrente. O sistema foi emulado no Mininet, sobre uma topologia em “diamante” com quatro roteadores, utilizando Scapy para medição de atraso unidirecional (One-Way Delay, OWD) de tráfego TCP uRLLC, iperf3 para geração de tráfego eMBB, e tc/HTB do Linux para priorização dinâmica de filas. Os resultados experimentais mostram que, sem nenhum mecanismo de controle, um enlace de transporte com banda limitada degrada a latência uRLLC de um baseline de $\approx 8\text{ms}$ para mais de 1,8s sob congestionamento eMBB (pico de 4,7s); com o Closed Loop dinâmico proposto, o mesmo cenário de congestionamento é detectado e mitigado automaticamente em poucos ciclos de controle, restaurando a latência ao nível de baseline.*

1. Introdução

Redes de quinta geração (5G) precisam suportar, na mesma infraestrutura de transporte, classes de tráfego com requisitos radicalmente diferentes. Aplicações do tipo *Ultra-Reliable Low Latency Communication* (uRLLC), por exemplo, controle remoto de veículos autônomos, exigem atraso fim-a-fim (*end-to-end*) inferior a 5ms, enquanto

aplicações do tipo *enhanced Mobile Broadband* (eMBB), como streaming de vídeo ou transferência de arquivos, demandam grande volume de dados, sem restrição rígida de latência [1].

Quando esses dois tipos de tráfego competem pelos mesmos enlaces de transporte, o tráfego eMBB pode saturar as filas dos roteadores e degradar severamente a latência do tráfego uRLLC, violando o requisito de 5ms. Este trabalho propõe e avalia um sistema de *Closed Loop*, um ciclo de monitoramento, decisão e atuação em tempo real, capaz de detectar essa degradação e reagir automaticamente, redistribuindo a capacidade dos enlaces de transporte entre as duas classes de tráfego para proteger o uRLLC. A topologia de rede utilizada também conta com dois caminhos redundantes entre origem e destino, o que abre espaço para uma segunda forma de atuação (redirecionamento de rota), tratada na Seção 2 como uma capacidade adicional já implementada, ainda que não seja o foco principal da avaliação.

O restante deste artigo está organizado da seguinte forma: a Seção 2 descreve a metodologia experimental, incluindo o ambiente de emulação, a topologia de rede e a evolução das técnicas utilizadas para geração e medição de tráfego; a Seção 3 detalha a proposta do mecanismo de *Closed Loop*; a Seção 4 apresenta a avaliação experimental, comparando os cenários sem controle, com QoS estático e com o Closed Loop dinâmico; e a Seção 5 apresenta as conclusões e trabalhos futuros.

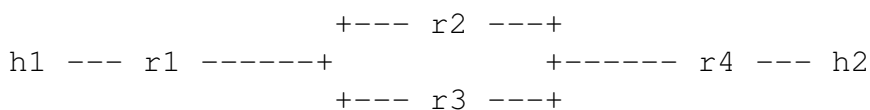
2. Metodologia

2.1. Ambiente de emulação

Todo o experimento foi emulado com o Mininet 2.3.0, executado em um notebook físico com Ubuntu 22.04 (kernel 6.8). O Mininet [2] cria *namespaces* de rede Linux isolados para cada nó da topologia, o que permite emular uma rede inteira dentro de um único kernel físico.

2.2. Topologia de rede

Foi construída uma topologia em “diamante” com quatro roteadores, representando a rede de transporte de uma operadora 5G:



h1 gera o tráfego uRLLC e eMBB, h2 atua como servidor de destino, e r1–r4 são hosts Linux com *IP forwarding* habilitado, configurados como roteadores via rotas estáticas (endereçamento /30 em cada enlace ponto-a-ponto). Os enlaces de acesso (h1↔r1 e r4↔h2) foram configurados com 100Mbit de banda, representando conexões de última milha não restritivas. Os enlaces de transporte (r1↔r2↔r4 e r1↔r3↔r4) foram limitados a 10Mbit, criando o gargalo onde o congestionamento entre uRLLC e eMBB de fato acontece. Optamos por limitar a banda em vez de inserir atraso artificial via parâmetro `delay`, para que o congestionamento observado viesse do comportamento real de fila do sistema operacional (via `tc/HTB`) e não de um número escolhido a dedo.

O roteador de entrada (r1) pode direcionar o tráfego por dois caminhos distintos: Rota A, via r2, ou Rota B, via r3. A função `set_active_route()` atualiza ao mesmo

tempo as rotas de ida (em $r1$) e de volta (em $r4$), preservando a simetria necessária para conexões TCP. Testamos essa troca de rota manualmente: com a Rota A ativa, o RTT medido entre $h1$ e $h2$ ficou em torno de 8,3ms; na Rota B, subiu para 16–20ms, já que os enlaces dessa rota têm maior contenção. A Tabela 1 resume essa comparação.

Tabela 1. Comparação entre as duas rotas da topologia diamante

Rota	Caminho	RTT (ping)	Uso no sistema
A	$r1 \rightarrow r2 \rightarrow r4$	$\approx 8,3$ ms	Rota ativa por padrão nos experimentos da Seção 4
B	$r1 \rightarrow r3 \rightarrow r4$	$\approx 16\text{--}20$ ms*	Rota alternativa; troca validada isoladamente, sem integração com o Closed Loop avaliado na Seção 4

rota. A primeira amostra da Rota B chegou a 28,7ms, provavelmente por causa da resolução ARP na primeira comunicação após a troca; as amostras seguintes ficaram entre 16 e 20ms.

Essa capacidade de troca de rota funciona e foi validada, mas não chegou a ser combinada com o mecanismo de priorização de filas nos testes finais (voltamos a esse ponto na Seção 3.3).

Durante o desenvolvimento apareceu um problema de roteamento assimétrico: $r2$ e $r3$ não tinham rota de volta configurada para a sub-rede de $h1$. O tráfego de ida chegava normalmente em $h2$, mas a resposta era descartada nesses roteadores intermediários por falta de rota, sem nenhuma mensagem de erro visível, o que tornou o diagnóstico mais demorado do que o esperado. A correção foi simples (adicionar rotas estáticas de volta em $r2$ e $r3$ apontando para $r1$), mas o episódio deixou claro um cuidado que não estava óbvio no início do trabalho: diferente de um switch de camada 2, um host Linux atuando como roteador não aprende rota nenhuma sozinho, então cada roteador intermediário precisa de rota explícita nos dois sentidos do tráfego, não só no sentido de ida.

2.3. Geração e medição do tráfego uRLLC

O tráfego uRLLC foi gerado e monitorado com Scapy [3], usando pacotes TCP, conforme pedido no enunciado. A forma de medir a latência mudou algumas vezes ao longo do desenvolvimento, até chegar na versão final:

1. **RTT via ICMP:** usamos *ping* numa primeira validação de conectividade da topologia. Serviu para os testes iniciais, mas não atende ao requisito de tráfego TCP do enunciado;
2. **RTT via TCP (SYN/SYN-ACK):** passamos a enviar pacotes TCP SYN com `Scapy.sr1()`, medindo o tempo até a resposta SYN-ACK. Na primeira versão, cada sonda abria e fechava um socket Scapy novo, o que deixava o RTT medido em 25–33ms. Reutilizando um único socket L3 (`conf.L3socket()`) para todas as sondas de uma execução, esse valor caiu para 13–16ms;
3. **OWD via TCP:** por fim, trocamos a métrica de RTT para atraso unidirecional (*One-Way Delay*), mais próximo do que o enunciado chama de atraso fim-a-fim.

A sonda em `h1` embute o *timestamp* de envio no *payload* de um pacote TCP e manda pelo mesmo socket L3 reutilizado, sem esperar resposta. Em `h2`, um processo Scapy captura os pacotes e calcula o OWD como a diferença entre o *timestamp* de chegada e o *timestamp* embutido no pacote.

Já dentro da abordagem de OWD, testamos também trocar a forma de captura em `h2`: em vez do `sniff()` do Scapy, usar diretamente um socket L2 de baixo nível (`conf.L2listen()`), pulando a etapa de compilação de filtro BPF que o `sniff()` faz por baixo dos panos. A Tabela 2 traz o resultado de cada uma dessas etapas.

Tabela 2. Evolução da latência de baseline medida ao longo das técnicas testadas

Etapa	Técnica	Baseline observado	Observação
1	ICMP / <i>ping</i>	≈8,4 ms (RTT)	Validação inicial de topologia; não usa TCP
2	TCP RTT, socket novo a cada sonda	≈25–33 ms (RTT)	Overhead de abrir/fechar socket a cada pacote (<code>sr1()</code>)
2b	TCP RTT, socket reutilizado	≈13–16 ms (RTT)	Reduziu quase pela metade em relação à etapa anterior
3	TCP OWD via <code>sniff()</code>	≈6–7 ms (OWD)	Elimina a espera por resposta do lado de quem envia
3b	TCP OWD via <code>L2listen()</code>	≈6–8 ms (OWD)	Praticamente igual à etapa anterior, sem ganho perceptível

A última linha da tabela é um resultado que esperávamos melhor: a troca para `L2listen()` não trouxe redução nenhuma que valesse a pena mencionar, o que sugere que o gargalo já não estava mais na função de captura, e sim em algo mais difícil de contornar sem sair do Python (agendamento de processo pelo Linux, custo do próprio interpretador). Voltamos a esse ponto na Seção 4.5.

Um detalhe que facilitou bastante a medição: como `h1` e `h2` são *namespaces* do mesmo kernel físico no Mininet, os dois compartilham o mesmo relógio de sistema. Isso elimina qualquer necessidade de sincronizar relógios via NTP entre origem e destino, o que só funciona porque estamos num ambiente emulado; numa implantação real e distribuída isso não valeria.

2.3.1. Limitação conhecida: overhead da ferramenta de medição

Mesmo depois de todas as otimizações acima, calibramos a rede sem nenhum congestionamento e ainda assim medimos uma latência de baseline entre 5ms e 8ms. Esse número é custo de processamento do Python/Scapy montando, enviando e capturando cada pacote (inclui o tempo que o *scheduler* do Linux leva para dar CPU ao processo), os enlaces

emulados em si têm propagação de poucos milissegundos, então essa latência não vem da rede. Por isso, usamos como métrica principal a degradação em relação ao baseline calibrado (latência atual menos baseline) em vez do valor absoluto, para separar o efeito do congestionamento do custo já conhecido da ferramenta de medição.

2.4. Geração de tráfego eMBB

O tráfego eMBB foi gerado com `iperf3`, simulando uma transferência de dados de alto volume (por exemplo, streaming ou transferência de arquivo) concorrente ao tráfego uRLLC, saturando o enlace de transporte de 10Mbit.

3. Proposta

3.1. Priorização de filas com HTB

O mecanismo de atuação principal usa *Hierarchical Token Bucket* (HTB), do subsistema `tc` do Linux. Em cada interface de saída da rota ativa, criamos duas classes filhas de uma classe raiz que representa a capacidade total do enlace (10Mbit):

- **Classe uRLLC** (1 : 10): prioridade 0, identificada por filtro `tc` que casa a porta de destino TCP 9000;
- **Classe eMBB** (1 : 20): prioridade 1, identificada por filtro que casa a porta de destino TCP 5201 (porta padrão do `iperf3`).

Os filtros usam o classificador `u32`, mapeando cada fluxo à sua classe de prioridade pela porta de destino, é o mecanismo de diferenciação de tráfego pedido no enunciado.

3.2. Closed Loop dinâmico

Diferente de uma configuração de QoS fixa, o mecanismo proposto muda a taxa de cada classe em tempo real, sem recriar a estrutura de filas, usando o comando `tc class change`. O controle opera em três níveis de atuação, definidos pelo par (taxa da fila eMBB, taxa da fila uRLLC), cuja soma é sempre igual à capacidade do enlace:

Tabela 3. Níveis de atuação do Closed Loop dinâmico

Nível	Taxa eMBB	Taxa uRLLC
0 (neutro)	5 Mbit	5 Mbit
1 (leve)	3 Mbit	7 Mbit
2 (forte)	1 Mbit	9 Mbit

A cada segundo, o loop de controle (executado em uma *thread* separada da sonda de medição) calcula a latência média das últimas cinco amostras de OWD e compara com o baseline calibrado no início da execução:

$$degradação = \overline{OWD}_{recente} - baseline$$

Quando a degradação passa do limiar de 5ms do enunciado, o sistema sobe um nível de atuação. Quando a degradação volta a ficar abaixo da metade do limiar, o sistema

recua um nível. Escolhemos esse escalonamento em degraus, em vez de uma resposta binária (ligado/desligado), justamente para não reagir com força total a uma degradação pequena e passageira. Cada mudança de nível fica registrada com *timestamp*, nível anterior/novo, latência média e degradação medida no momento da decisão.

3.3. Camada complementar de atuação: redirecionamento de rota

Além da priorização de filas, a topologia em diamante (Seção 2.2) permite uma segunda forma de atuação: mandar o fluxo uRLLC pela Rota A ou pela Rota B via `set_active_route()`, caso a degradação continue mesmo com a fila já no nível máximo de prioridade. Essa troca de rota funciona e foi validada isoladamente, mas não entrou nos experimentos da Seção 4, decidimos avaliar cada mecanismo separado, para não misturar o efeito de um com o do outro nos números finais. Juntar os dois em um único controlador (priorizar fila primeiro, e só trocar de rota se isso não bastar) fica como próximo passo, discutido na Seção 5.

4. Avaliação

4.1. Cenário sem controle (baseline do problema)

Com a estrutura de filas padrão, sem priorização nenhuma, disparamos uma rajada de tráfego eMBB de 20 segundos depois de um período de aquecimento. Os resultados:

Tabela 4. Resultados do cenário sem QoS

Métrica	Valor
Latência média antes do eMBB	7,41 ms
Latência média durante o eMBB	1881,46 ms
Latência média depois do eMBB	1274,61 ms
Pico máximo de latência	4733,32 ms

O crescimento quase linear da latência durante o congestionamento é o clássico *bufferbloat*: uRLLC e eMBB dividem a mesma fila FIFO, então um pacote uRLLC que chega atrás de uma rajada de pacotes eMBB tem que esperar toda a fila esvaziar antes de ser processado, e essa fila só cresce enquanto o eMBB continua saturando o enlace.

4.2. Cenário com QoS estático

Com a estrutura de filas com prioridade (Seção 3.1) ligada o tempo todo, sob o mesmo padrão de tráfego eMBB:

Tabela 5. Resultados do cenário com QoS estático

Métrica	Valor
Latência média antes do eMBB	7,61 ms
Latência média durante o eMBB	6,94 ms
Latência média depois do eMBB	6,68 ms
Pico máximo de latência	29,61 ms

A latência durante o congestionamento caiu de 1881,46ms para 6,94ms, cerca de 270 vezes menor, e o pico caiu de 4733,32ms para 29,61ms, uns 160 vezes menor. Também apareceram mais retransmissões TCP no fluxo eMBB (visível na saída do iperf3), o que sugere que a própria priorização faz o TCP do eMBB reduzir sua taxa de envio sozinho, já que sobra menos espaço de fila disponível para essa classe.

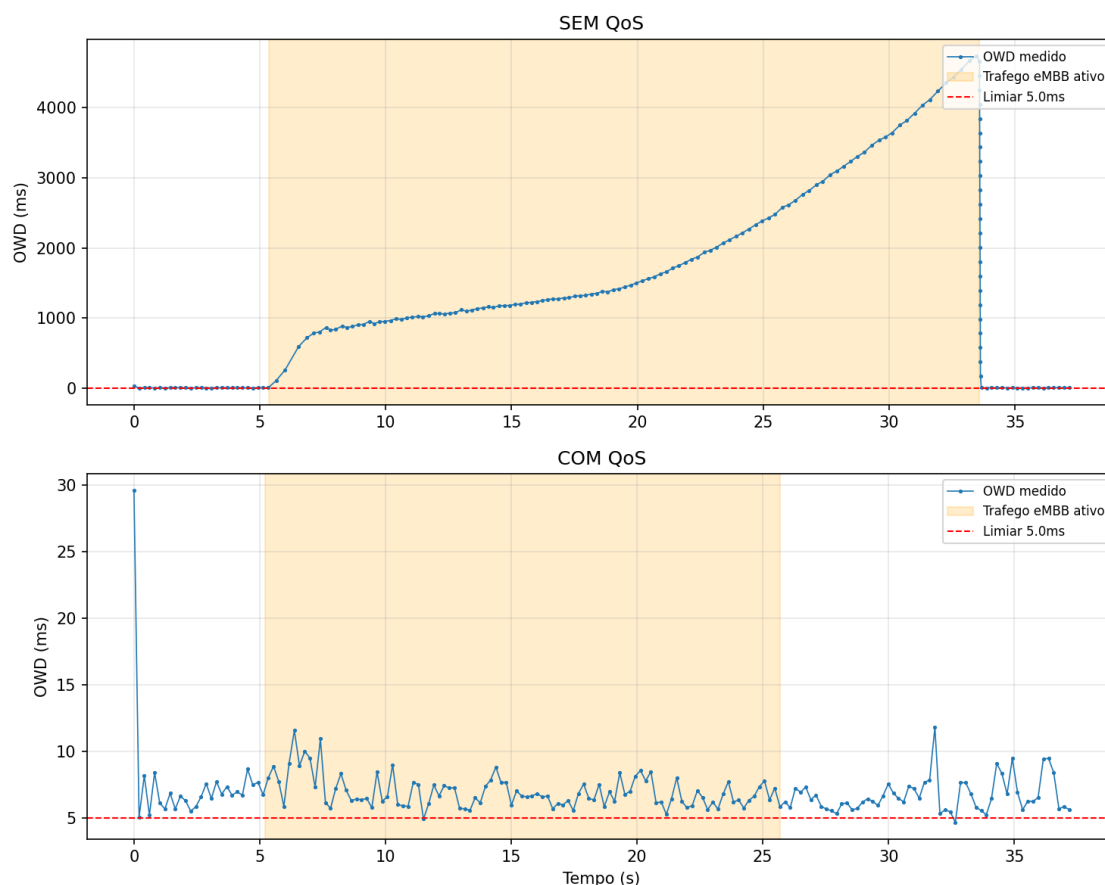


Figura 1. Latência OWD medida ao longo do tempo nos cenários sem QoS (acima) e com QoS estático (abaixo). A área sombreada marca a janela de tráfego eMBB ativo; a linha tracejada marca o limiar de 5ms. Repare na escala do eixo vertical: o painel de cima chega a 4700ms, o de baixo não passa de 30ms.

A Figura 1 mostra bem o que as duas tabelas já contavam em número. No cenário sem QoS, a latência sobe o tempo inteiro enquanto o eMBB está ativo, sem dar sinal de estabilizar, e assim que o eMBB para, por volta dos 33,7s, a latência cai de volta pra perto de zero quase instantaneamente. Isso mostra que a fila esvazia rápido assim que a causa do congestionamento some; o problema não deixa sequelas, é um efeito direto do tráfego concorrente enquanto ele dura. Já no cenário com QoS estático, a latência fica presa numa faixa estreita (entre 5ms e 12ms, aproximadamente) do começo ao fim, com só um pico de aquecimento inicial de 29,6ms (o mesmo efeito de primeira comunicação já comentado na Seção 2.3). O tráfego eMBB, mesmo presente na mesma janela de tempo, praticamente não deixa marca visível no padrão de latência do uRLLC.

4.3. Cenário com Closed Loop dinâmico

No experimento com o controle dinâmico completo (Seção 3.2), o sistema calibrou um baseline de 8,27ms antes do eMBB começar. Durante a rajada de congestionamento, o sistema reagiu sozinho:

Tabela 6. Transições de nível do Closed Loop dinâmico durante o experimento

Transição	Latência média	Degradação	Motivo
Nível 0 → 1	25,13 ms	16,86 ms	degradação > limiar de 5ms
Nível 1 → 2	25,13 ms	16,86 ms	degradação > limiar de 5ms
Nível 2 → 1	8,41 ms	0,14 ms	degradação normalizada
Nível 1 → 0	8,41 ms	0,14 ms	degradação normalizada

O sistema subiu dois níveis diante da degradação inicial, e assim que a latência voltou perto do baseline (degradação de só 0,14ms), recuou sozinho até o nível neutro, sem ninguém mexer em nada, é o ciclo de monitorar, decidir e agir que o enunciado pede.

A Tabela 7 junta os três cenários lado a lado.

Tabela 7. Comparação consolidada entre os três cenários avaliados

Métrica	Sem QoS	QoS estático	Closed Loop dinâmico
Latência média antes do eMBB	7,41 ms	7,61 ms	8,27 ms
Latência média durante o eMBB	1881,46 ms	6,94 ms	25,13 ms
Latência média depois do eMBB	1274,61 ms	6,68 ms	8,41 ms
Pico máximo de latência	4733,32 ms	29,61 ms	,-

Vale explicar por que essa tabela mistura números que não são exatamente do mesmo tipo: nos dois primeiros cenários, cada valor é a média sobre toda a série de amostras coletadas dentro da respectiva janela de tempo (antes/durante/depois). Já no Closed Loop dinâmico, os valores de "durante" e "depois" vêm dos instantes registrados nas transições de nível (Tabela 6), não uma média contínua da mesma forma, e por isso o pico máximo do Closed Loop não está preenchido: o sistema não guarda esse dado isolado, só os valores de latência no momento de cada decisão. Ainda assim, a comparação deixa claro que o mecanismo dinâmico mantém a degradação numa faixa parecida com a do QoS estático, sem precisar de uma política fixa o tempo todo.

4.4. Discussão

Os três cenários mostram, um depois do outro, o valor de cada parte da proposta. Sem nenhum controle, dividir a mesma fila entre uRLLC e eMBB simplesmente não sustenta o requisito de 5ms. Com priorização fixa, a degradação cai bastante, mas a política fica ligada o tempo inteiro, mesmo quando não há congestionamento algum acontecendo. Com o Closed Loop dinâmico, o sistema só prioriza quando precisa, na intensidade que precisa, e desliga a priorização sozinho quando o problema passa, esse é o comportamento mais próximo do que o enunciado pede. A troca de rota (Seção 3.3), apesar de funcionar, não entrou em ação em nenhum desses experimentos, já que a priorização de fila sozinha já deu conta de segurar a degradação dentro do limiar avaliado.

Nem tudo saiu como planejado da primeira vez, e vale registrar isso. A topologia diamante só ficou estável depois de corrigirmos o problema de roteamento assimétrico (Seção 2.2), antes disso, a conectividade entre $h1$ e $h2$ falhava de um jeito que não era óbvio de diagnosticar. Outro ponto que custou mais tempo do que imaginávamos no início foi coordenar o experimento inteiro (subir a topologia, aplicar QoS, iniciar sonda e servidor, disparar o eMBB, coletar os dados) sem intervenção manual. Uma primeira tentativa, digitando os mesmos passos em terminais separados e cronometrando o início do eMBB “no olho”, produziu dados contaminados: sobrava amostra de execução anterior misturada por arquivo de log não limpo, e a janela de tempo do eMBB não batia direito com o período real de coleta da sonda, por causa do tempo gasto entre um comando e outro. Automatizar tudo isso num único script Python resolveu o problema de vez, e foi provavelmente o ajuste de maior impacto prático em todo o trabalho, não por ser sofisticado, mas por cortar de raiz uma fonte grosseira de erro experimental.

Também tentamos reduzir ainda mais o *overhead* de medição trocando o `sniff()` do servidor por um socket L2 de baixo nível do próprio Scapy (`conf.L2listen()`), na expectativa de que pular a camada de conveniência do `sniff()` (compilação de filtro BPF, gerenciamento de buffer) ajudasse. O resultado ficou praticamente igual ao anterior, entre 6ms e 8ms, com picos ocasionais de até 12ms, ou seja, essa otimização específica não valeu o esforço. O gargalo já não estava mais na função de captura, e sim em algo mais difícil de contornar sem sair do Python, como o agendamento de processo feito pelo Linux e o custo do próprio interpretador para desserializar cada pacote. Registramos esse resultado negativo porque ele delimita até onde dá pra otimizar dentro da abordagem escolhida.

Mesmo depois de todas essas otimizações, não conseguimos fazer a latência medida em valor absoluto ficar de forma confiável abaixo dos 5ms do enunciado, o baseline calibrado oscilou entre 5ms e 8,3ms conforme a execução, mesmo com a rede em condição ideal, sem congestionamento nenhum. Essa é uma limitação da instrumentação, não da rede em si, e por isso adotamos a degradação relativa ao baseline como critério principal de avaliação (Seção 2.3.1). Fica registrado, de qualquer forma: uma leitura isolada de uma amostra de latência do sistema, sem olhar o baseline, pode dar a entender que o requisito de 5ms foi violado mesmo numa rede perfeita.

Duas outras limitações merecem ser ditas com a mesma clareza. Os resultados da Seção 4 vêm de uma única execução de cada cenário, não de médias de várias repetições, os números absolutos (os 4733,32ms de pico do cenário sem controle, por exemplo) podem variar de uma execução para outra por fatores do sistema operacional que não controlamos, ainda que a ordem de grandeza do efeito (a diferença de duas a três ordens de magnitude entre ter ou não priorização) se repetiu em execuções exploratórias feitas ao longo do desenvolvimento. E, como já mencionado, a camada de redirecionamento de rota (Seção 3.3) não chegou a ser integrada ao Closed Loop dinâmico na versão avaliada: decidimos testar apenas a priorização de filas para não misturar o efeito dos dois mecanismos, mas isso significa que o sistema completo, com as duas camadas juntas, ainda não passou pela mesma validação experimental do resto.

Dado isso tudo, como interpretar os resultados da Seção 4 com os pés no chão? A melhora observada (cerca de 270 vezes na latência média durante congestionamento, e 160 vezes no pico) é grande e bate com o que se espera de um mecanismo de priorização

de fila sob um gargalo de banda bem definido, o *bufferbloat* eliminado pela priorização é um fenômeno já bem descrito na literatura de redes, e os dados coletados aqui são compatíveis com isso. O comportamento do Closed Loop dinâmico (subindo dois níveis e recuando outros dois, de forma proporcional à degradação medida) também é consistente com o algoritmo implementado na Seção 3.2, e mostra que o ciclo de monitorar-decidir-agir funciona nesse experimento específico. Por outro lado, com o número de execuções que temos, não dá para afirmar com confiança estatística como o sistema se comportaria sob outros padrões de tráfego eMBB (rajadas mais curtas e frequentes, ou vários fluxos eMBB ao mesmo tempo, por exemplo), nem quantificar o quanto o tempo de reação do Closed Loop varia entre execuções diferentes. Essas questões ficam como direção concreta para dar continuidade a este trabalho.

5. Conclusões

Este trabalho apresentou o projeto e a avaliação de um sistema de monitoramento e controle *Closed Loop* para tráfego uRLLC em redes de transporte 5G, emulado em Mininet com geração/medição de tráfego via Scapy e iperf3, e atuação via priorização dinâmica de filas HTB no Linux. Os resultados mostram que o mecanismo proposto reduz a degradação de latência sob congestionamento em duas ordens de grandeza em relação ao cenário sem controle, e que o controle dinâmico é capaz de detectar, reagir e recuar automaticamente conforme as condições da rede mudam.

Como limitação principal, o overhead de medição do Scapy em Python (Seção 2.3.1) impede checar o requisito de 5ms em valor absoluto, sendo necessário avaliar a degradação relativa a um baseline calibrado. Como trabalhos futuros, pretendemos: (i) repetir os experimentos estatisticamente (várias execuções) para reportar intervalos de confiança; (ii) integrar as duas camadas de atuação já implementadas, priorização de filas e redirecionamento de rota, em um único controlador que só troca de rota quando a priorização de fila no nível máximo não for suficiente; (iii) investigar a migração do plano de dados para P4/BMv2, o que eliminaria o overhead de medição em Python ao mover a marcação de tempo para o próprio plano de dados.

Referências

- [1] ITU-T. *IMT Vision, Framework and overall objectives of the future development of IMT for 2020 and beyond*. Recommendation ITU-R M.2083-0, 2015.
- [2] LANTZ, B.; HELLER, B.; McKEOWN, N. *A Network in a Laptop: Rapid Prototyping for Software-Defined Networks*. In: *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*, 2010.
- [3] BIONDI, P. *Scapy: Packet Manipulation Tool*. Disponível em: <https://scapy.net/>. Acesso em: 31 jul. 2026.